WEEK 2 CASE ANALYSIS

Myntra: From a Fixed-Capacity Monolith to Cloud-Native Flash-Sale Scaling

Course: Cloud Computing – Case Study Project

Case Study: Cloud-Based Personalization and Flash-Sale Scaling

Focus Area: Scalability and High-Traffic Events

Group Number: 33

Team Members

- **Utsker Dhameliya – 202301083**
- **Ritul patel – 202301086**
- **Divyesh vagh – 202301444**

Submission: Week 2 Deliverable

Contents

- Case Analysis and Problem Definition
- Existing and Proposed Architecture Diagrams
- Scalability Challenges and Proposed Solutions
- Implementation and Exercise Plan
- Team Roles and Responsibilities

Department: Information and Communication Technology (ICT)

Institution: Dhirubhai Ambani University (DAU)

Academic Year: 2026–2027

1. Executive Summary

Myntra is one of India's largest fashion e-commerce platforms. Its business is shaped by short, extreme traffic events: the End of Reason Sale (EORS) and the Big Fashion Festival (BFF). During such events demand rises far above normal and the company's revenue depends on every layer of the system staying up.

This report studies how Myntra's technology moved from an early, fixed-capacity monolith (PHP application servers talking directly to a MySQL database) that collapsed under sale traffic, to a service-oriented, cloud-based platform with real-time data, personalization and elastic infrastructure on Microsoft Azure. We describe the old architecture, the problems it created, the solution Myntra adopted, the resulting architecture, and the measurable outcomes Myntra has published.

We then propose a Week 3 exercise that reproduces the core idea at small scale: a fixed-capacity deployment versus an autoscaled deployment, subjected to a flash-sale load test, comparing latency, scaling behaviour and cost.

Key finding. The central failure in the early system was not raw lack of servers but a cascading failure: when database calls slowed, application servers stopped accepting requests, the load balancer removed them from the pool, and the remaining servers were overloaded in a chain reaction [1]. The solution combined asynchronous request handling, independently scalable services, and later an elastic cloud data platform. The lesson is that elasticity must be designed into the whole stack, not only added at the server layer.

2. Introduction and Context

2.1 About the company

Myntra is an Indian fashion and lifestyle e-commerce company, part of the Flipkart group. It runs large annual sale events, with EORS in the middle and at the end of the year and BFF around the festive season.

2.2 Why this case is a good fit for cloud computing

The workload has the classic cloud-friendly shape:

- **Spiky demand:** traffic during a flash sale is a large multiple of normal traffic, and lasts hours, not weeks.
- **Revenue-critical availability:** downtime during a sale directly loses sales and customer trust.
- **Data-intensive personalization:** recommendations, search suggestions, coupons and ads depend on processing user behaviour in near real time.
- **Cost pressure:** provisioning for the peak all year is wasteful, so elasticity matters.

2.3 Scope and method

This study is based on primary and near-primary sources: Myntra's own engineering and corporate blogs and engineer-authored posts (see References). Where the sources do not state a detail (for example exact server counts, exact cloud bills or the precise date of each migration step) we say so rather than guess.

2.4 Research questions

1. What did Myntra's early architecture look like, and where did it fail under flash-sale load?
2. What changes did Myntra make, and why?
3. What does the resulting architecture look like and what outcomes are reported?
4. How can we reproduce the key behaviour in a small experiment?

3. The Legacy Architecture (Before)

3.1 Description

According to an engineer who joined in 2013, Myntra's stack at that time was a monolith written in PHP that spoke directly to MySQL [1]. This stack was successful enough to take the company to a point where a rewrite became possible, but it was built around a fixed set of application servers sitting behind a load balancer. [The source refers to these as PHP machines; it does not say whether they were physical servers or virtual machines. In our diagram we treat them as fixed-capacity servers, which is the same scaling model as a traditional VM fleet. Confirm wording with the instructor if needed.]

3.2 Request flow

1. A customer's browser or app sends a request to the website.
2. A load balancer distributes requests across a fixed pool of PHP application servers.
3. Each PHP server handles the entire request (catalogue pages, cart, checkout, pricing and so on) and makes synchronous calls to the MySQL database.
4. The server waits for the database result before replying.

Figure 1 – Legacy architecture (to be redrawn in draw.io / Excalidraw):

```
 Customers (web / app)
          |
  [ Load Balancer ]
    |     |     |
[PHP] [PHP] [PHP] ... (fixed pool, ~10+ servers)
  \     |     /
   [ MySQL database ]   <- single shared bottleneck
```

3.3 Characteristics of this design

Aspect	Legacy design
Application style	Single PHP monolith: all features deployed and scaled together
Scaling model	Add more servers manually; fixed capacity decided in advance
Request handling	Synchronous: each request occupies a server while waiting for the database
Data layer	Shared MySQL database directly queried by the application
Failure behaviour	Failure of some servers shifts load to the rest, which then also fail
Analytics and personalization	Batch jobs over previous-day data (the data platform article says personalization was batch-based until 2021) [2]

4. The Problems

4.1 Cascading failure under sale traffic

The engineer's account describes sales as a financial dream but a technology nightmare. Once traffic passed a threshold, PHP servers were waiting on so many database requests that they stopped accepting new ones. The load balancer then removed those servers from the pool as unhealthy, pushing the same traffic onto fewer servers, which also failed. This chain reaction could take down the entire stack, and recovery meant a reboot [1].

4.2 Poor capacity efficiency

Another Myntra engineer notes that more than ten PHP machines would go down whenever a sale ran, and that the company struggled to handle on the order of a crore of rupees of sales per day in that period. After the Node.js front-end launched, all of Myntra's traffic could be run through a single server [3]. This shows how much capacity was being lost to blocking behaviour rather than genuine lack of hardware.

4.3 Tight coupling and monolithic scaling

Because everything shipped together, a spike in one feature (for example product listing) meant scaling the whole application, and a failure in one path (for example database slowness) affected all paths. [Team: this is our analysis of the monolith pattern, supported by the source's own description of the PHP stack.]

4.4 Data platform limits

The later data platform article describes a second class of problem: analytics and serving systems that could not keep up. An early self-serve query platform (DDP) built on a warehouse suffered frequent

out-of-memory issues and a median query wait above seven minutes. Storage was coupled with compute in the warehouse, so during high-rate days the compute alone could not be scaled. Query volume was growing about 60% year over year [2].

4.5 Batch-only personalization

Personalization and recommendations ran as batch jobs until 2021 [2]. In fashion e-commerce a user's intent can change within a session, so day-old data limits relevance during exactly the events when relevance matters most.

4.6 Summary of problems

#	Problem	Evidence	Business impact
1	Cascading failure from blocking DB calls	Servers stopped accepting requests and were removed from the pool; recovery by reboot [1]	Outages during the highest-revenue events
2	Wasted capacity	10+ servers failing during sales; one Node server later handled all traffic [3]	High hardware need and poor reliability
3	Monolith coupling	PHP monolith with direct MySQL access [1]	Cannot scale features independently
4	Storage and compute coupling in analytics	ADW coupling; OOM errors; 7+ minute median waits [2]	Slow insight and costly scaling
5	Batch personalization	Batch until 2021 [2]	Less relevant recommendations in-session

5. The Solution

Myntra's response was not a single migration but a sequence of changes across the front end, the services layer, the cloud platform and the data platform.

5.1 Step 1 – Asynchronous front end on Node.js

The team considered rewriting the PHP stack to be asynchronous, or fronting the database with microservices that scale on their own. Since a rewrite was already needed, they chose a framework where asynchronous handling is the default and picked Node.js [1]. The engineer's account says the new layer was built in about six months, that it covered the whole stack from servers and service clients to builds and an isomorphic browser application, and that fixing this layer let the backend services finally receive the load they were designed for [3].

5.2 Step 2 – Dedicated microservices

In parallel, teams were building dedicated services corresponding to site features, so that the front end had independently scalable backends to call [3]. Myntra's engineering blog also has a series on scalable microservice architecture (for example the IDEA 2.0 post) [4].

5.3 Step 3 – Cloud platform with open-source foundations

Myntra's data platform article states that the company relies heavily on Microsoft Azure as a partner while keeping open-source foundations across the layers of its stack, which lets each layer choose the best-suited tool [2]. The platform was built in-house over several years using open-source technologies and launched in 2020 with petabyte-scale real-time analytics capability [2].

5.4 Step 4 – Elastic, decoupled data platform

The data platform article describes these building blocks [2]:

- **Sourcerer:** ingestion of structured, unstructured and clickstream data.
- **Quicksilver:** near-real-time streaming platform that turns raw events into insights and ML actions.
- **Janus:** in-house cluster orchestrator and data-processing framework for petabyte-scale processing in the data lake.
- **Bifrost:** serving layer, a lakehouse built on Apache Superset and Trino, which replaced the warehouse-based serving path. Compute and storage were separated, and Hive ACID was introduced for mutable transactional data.
- **Kafka (clickstream):** deployed in-place scaling to handle high traffic while saving cost.
- **Druid:** real-time analytics database that gave sub-second queries on real-time data, replacing a slower approach that used MemSQL.

5.5 Step 5 – Real-time personalization and customer engagement

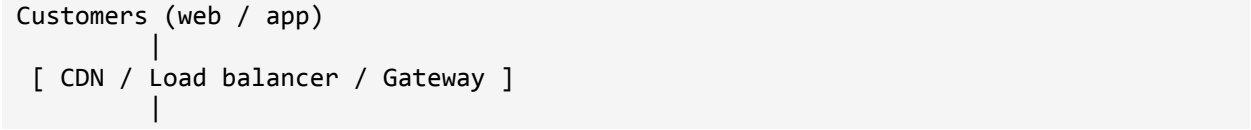
A Near Real Time (NRT) platform was created to capture user actions in real time and use them for personalized UX, recommendations, search results and coupons [2]. Use cases include similar-product and cross-sell recommendations, ad ranking with frequency capping, and personalized search auto-suggest [2]. A customer data platform called Personify handles segmentation and campaigns [2].

6. The New Architecture (After)

6.1 Description

The modern Myntra platform is service-oriented with an elastic cloud and an event-driven data layer. We present a logical view based on the components named in Myntra's own posts. [Team: this is a simplified logical diagram for study purposes, not an official Myntra diagram. Label it as such in the final report.]

Figure 2 – Modern architecture, logical view (to be redrawn):



```
 [ Node.js front-end / BFF layer ]
          |
 +-------+--------+---------+----------+
 |       |        |         |          |
[Catalogue] [Cart] [Orders] [Search] [Pricing] ...  <- independent microservices
 |       |        |         |          |
[Service datastores / caches]

 (every user action = an event)
          |
 [ Kafka clickstream ] -> [ Quicksilver NRT ] -> personalization,
          |                       |               recommendations, ads, coupons
          v                       v
 [ Data lake (Janus) ]   [ Druid real-time analytics ]
          |
 [ Bifrost lakehouse: Trino + Superset ] -> analysts, dashboards, ML

 Hosted on: Microsoft Azure + open-source stack
```

6.2 How the new design addresses each problem

Problem	Fix in the new design
Cascading failure from blocking DB calls	Asynchronous Node.js layer; services scale independently [1][3]
Wasted capacity	Far higher throughput per server; elastic cloud capacity [3][2]
Monolith coupling	Dedicated microservices per feature [3][4]
Storage and compute coupling	Lakehouse with separated storage and compute (Trino) [2]
Batch personalization	Near-real-time platform with Kafka and Druid [2]

7. Outcomes and Reported Data

The figures below come from Myntra's data platform article about BFF 2022 and EORS [2] unless otherwise marked. Verify each number against the original before submission.

Metric	Reported outcome
Peak events (app and web clicks and page loads)	18.8 billion on day one of BFF 2022, up 13% year over year
Data served	2.5 petabytes per day by the serving layer during BFF 2022, with more than 65,000 queries per day (30%+ increase over the mid-year EORS)

Metric	Reported outcome
Customer engagement	Combined peak of 2 million sessions for customer-engagement and personalization during BFF
Real-time analytics efficiency	Druid achieved the same aggregations as the old approach with roughly 40% less compute, and results in milliseconds instead of seconds to minutes
Segmentation efficiency	About 50% lower average turnaround vs 2021; 8x more segments with only a 2x increase in underlying infrastructure
Reliability	99.9% segment completion SLA on the Personify platform
Query serving	Phased path to serving data in as little as 15 seconds, versus 7+ minute median waits in the earlier platform
Front-end throughput (earlier era)	After the Node.js launch, all Myntra traffic could run through one server, versus 10+ PHP machines going down during sales [3]

Cost data. The sources do not publish Myntra's actual cloud spend. The cost evidence available is relative (about 40% less compute for Druid; 8x segments for 2x infrastructure). In Week 3 we will produce our own cost model and measurements for the experiment, and we will clearly label them as our estimates rather than Myntra's figures.

8. Analysis and Lessons Learned

1. **Elasticity needs a non-blocking architecture.** Adding servers does not help if each server blocks on a shared database; the failure mode was a cascade, not simple shortage.
2. **Decouple what scales differently.** Separate services and separate storage and compute let each part scale to its own demand.
3. **Real-time data changes the product.** Moving from batch to near-real-time personalization is a cloud-enabled capability that directly affects relevance and revenue during sales.
4. **Use open-source on cloud for flexibility.** Myntra mixes Azure with open-source tools rather than locking each layer to a single vendor product [2].
5. **Efficiency matters as much as scale.** Several reported gains are about doing more with less (40% less compute, 8x segments with 2x infrastructure), which is the cost side of cloud elasticity.
6. **Limits of this study.** The public sources are engineering blog posts written by Myntra staff; they emphasize successes, and they do not disclose bills, instance counts or incident post-mortems.

9. Exercise Plan (Week 3)

This follows the scalability exercise: deploy a small web app behind a load balancer with autoscaling, run a load test, and plot response time against instance count.

9.1 Objective

Show experimentally that a fixed-capacity deployment (our stand-in for the legacy model) degrades or fails under a flash-sale spike, while an autoscaled deployment (our stand-in for the cloud-native model) keeps latency acceptable, and quantify the cost difference.

9.2 Hypotheses

- H1: Under a sudden spike, the fixed deployment shows rising p95/p99 latency and errors, while the autoscaled deployment stays within a latency target after a short scale-out delay.
- H2: The autoscaled deployment costs less than provisioning the fixed deployment for peak all day, for a spike that lasts a short time.

9.3 System under test

- A small web application with two endpoints: a catalogue endpoint (read-heavy) and a cart/checkout endpoint that touches a database, to mimic the blocking-database pattern from the case. [Choose Python Flask or Node.js.]
- A database or managed database service (small tier).
- Packaged in Docker.

9.3.1 Two configurations

	Configuration A – Fixed (legacy-style)	Configuration B – Autoscaled (cloud-native style)
Compute	Fixed number of instances sized for normal load	Same instance type with autoscaling (for example VM scale set on Azure, or Kubernetes HPA)
Load balancer	Yes	Yes
Scaling rule	None	Scale out on CPU or request rate threshold; scale in after cool-down

We plan to use Azure because the case company relies on it, subject to student credits and quotas. A local alternative is Docker with Minikube or kind. [Team decision: cloud provider and region.]

9.4 Load test design

Tool: k6 or Locust.

1. **Baseline:** steady normal traffic for 5 minutes.
2. **Ramp:** gradually raise to the flash-sale level.
3. **Spike:** sudden jump to several times baseline for 10 minutes (mimicking a sale start).
4. **Recovery:** return to baseline and observe scale-in.

9.5 Measurements to collect

Metric	How collected
Latency p50, p95, p99	Load-test tool output
Throughput (requests per second) and error rate	Load-test tool output
Instance count over time	Cloud monitoring or Kubernetes metrics
Time to scale (spike start to new capacity serving)	Timestamps from monitoring
Cost	Provider pricing calculator plus actual billing for the test window; cost per hour and per 1,000 requests

9.6 Expected outputs

- Plot of response time against instance count (required by the instructor).
- Plot of latency and instance count over time during the spike.
- Table comparing A and B on latency, errors, time-to-scale and cost.
- Short analysis linking the findings back to Myntra's cascading-failure story.

9.7 Risks and mitigations

Risk	Mitigation
Cloud credits run out	Use the smallest instance sizes; shut resources down after each run; set budget alerts
Results vary between runs	Repeat each test at least three times and report the range
Load generator becomes the bottleneck	Run the load generator on a separate machine and monitor its CPU
Quota limits block scale-out	Request or check quotas before Week 3

10. Roles of Team Members

[Adjust to the actual group size; one person can hold more than one role.]

Role	Responsibilities	Name
Project lead and integrator	Plan, deadlines, merging the report, viva coordination	[Name]
Research and sources	Collect and verify Myntra sources, keep the reference list and data table	[Name]

Role	Responsibilities	Name
Architecture and diagrams	Draw Figures 1 and 2, keep diagrams consistent with the text	[Name]
Problem and solution analyst	Sections 4 to 8, comparison tables and lessons learned	[Name]
Cloud and DevOps engineer	Build the app, containers, deployment, autoscaling configuration	[Name]
Load testing and cost analyst	Test scripts, data collection, pricing calculator, graphs	[Name]

11. Project Timeline

Week	Plan
Week 2 (this report)	Case analysis, diagrams, exercise plan, roles
Week 3	Build the app and both configurations, run experiments, collect latency, scaling and cost data, add results to the report
Week 4	Final report editing, formatting and submission
Week 5	Viva preparation: each member can explain the architecture, the problem, the fix and the experiment

12. Conclusion

Myntra's story shows that the hardest part of scaling for a flash sale is removing hidden bottlenecks. The early PHP monolith with synchronous database calls failed in a cascade under sale traffic. Myntra answered with an asynchronous front end, dedicated microservices, a cloud platform based on Azure and open-source tools, and a real-time, decoupled data platform that supports personalization at very large event volumes. Published outcomes include billions of events per day, petabytes served daily, and large efficiency gains. Our Week 3 experiment will test the central idea at small scale by comparing a fixed deployment with an autoscaled one under a simulated flash sale.

References

[1] Myntra engineering: "Myntra: A Front-end Tech Story", Myntra Engineering on Medium. https://medium.com/myntra-engineering/myntra-a-front-end-tech-story-ded7d3626f6

[2] Myntra engineering: "How Myntra's Data Platform drives a seamless shopping experience for customers during its marquee events", Myntra Engineering on Medium, 26 May 2023. https://medium.com/myntra-engineering/how-myntras-data-platform-drives-a-seamless-shopping-experience-for-customers-during-its-marquee-310e47accd67

[3] Myntra engineering blog: "Personal thoughts on the shutdown of the myntra.com website". https://myntra.github.io/

[4] Myntra Engineering, Microservices tag, including "IDEA 2.0 – A look at scalable Micro-Service Architecture". https://medium.com/myntra-engineering/tagged/microservices

[5] Myntra blog: "A peek into Myntra's technology preparedness for the End Of Reason Sale (EORS)", 21 December 2021.
https://blog.myntra.com/a-peek-into-myntras-technology-preparedness-for-the-end-of-reason-sale-eors / [Team: read the full article, our tool could not load its body text, and add its points to Sections 5 to 7.]

[Add: Week 1 sources, Azure pricing calculator, k6 / Locust documentation.]